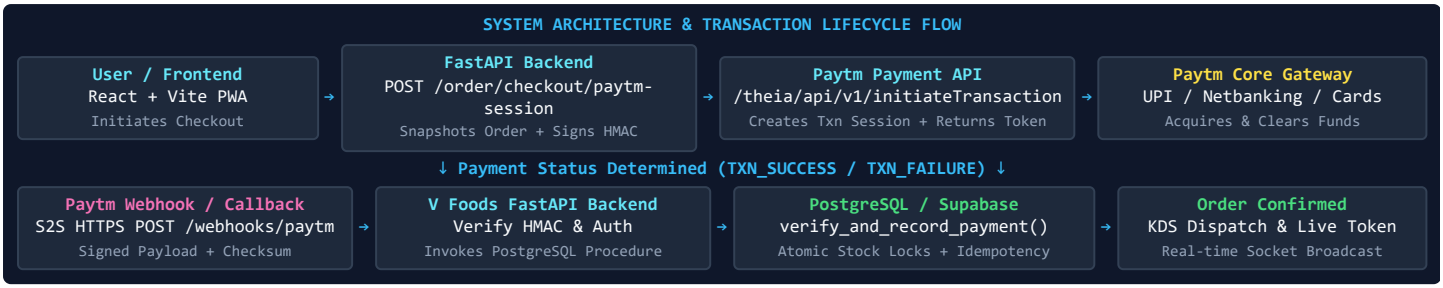
Page 1 — Complete Payment Architecture

End-to-End Payment Processing Pipeline, Gateway Orchestration & State Transitions



1. Where Payment Starts & Intake API

ACTUAL IMPLEMENTATION

Payment starts when a customer selects food items and taps **"Pay via Paytm"** in the client application. The frontend issues an authenticated HTTP POST request to:

`POST /order/checkout/paytm-session`

- **Validation:** Backend validates authenticated user (`user_id`), outlet active state, operating hours, and cart item availability.
- **Pessimistic Order Snapshot:** Inserts a row into `orders` with status `awaiting_payment` and inserts line items into `order_items` .
- **Payment Record Initialization:** Inserts a corresponding row into `payments` with status `PENDING` , linking `order_id` and `amount` .

2. Communicating with Paytm & Token Return

PAYTM OFFICIAL

The backend acts as the secure mediator to avoid exposing credentials:

- **Payload Generation:** Prepares order payload containing Merchant ID (`PAYTM_MID`), unique `orderId` , transaction amount, and callback URL.
- **Cryptographic Signing:** Calculates HMAC-SHA256 signature using the secret `PAYTM_MERCHANT_KEY` .
- **Paytm Response:** Paytm returns a transaction token (`txnToken`). The backend forwards this token, `orderId` , and `amount` to the client to initialize Paytm Checkout JS / Custom UI.

3. Gateway Communication Back & Verification

ACTUAL IMPLEMENTATION

When payment completes or terminates at Paytm:

- **Primary Channel (Webhook):** Paytm sends an asynchronous HTTPS POST request directly to `POST /webhooks/paytm` . This server-to-server (S2S) notification contains the gateway reference (`TXNID`), status (`STATUS: TXN_SUCCESS`), and response signature.
- **Checksum Verification:** FastAPI executes `_paytm_verify_checksum()` to prevent payload spoofing before processing.
- **Fallback Polling:** If the webhook is delayed or dropped, the client or background reconciliation queries `POST /admin/payments/reconcile` calling Paytm's `/v3/order/status` API.

4. State Persistence & Order Confirmation

ACTUAL IMPLEMENTATION

Final authority resides in the database, never the client:

- **Atomic Stored Procedure:** Backend calls PostgreSQL procedure `verify_and_record_payment()` .
- **Storage:** Status is recorded in `payments.status` (`SUCCESS`), recording `paytm_txn_id` and `paid_at = NOW()` .
- **Stock Decrement:** Acquired row locks (`FOR UPDATE`) decrement item stock atomically.
- **Order Confirmation:** Order status transitions from `awaiting_payment` to `confirmed` , generating the kitchen token for KDS tracking.

Architectural Principles & Guarantees

Zero Client Trust: The client application is strictly treated as an untrusted presentation layer. Only cryptographically verified S2S webhooks or direct backend-to-Paytm queries can confirm payments.

Strict Idempotency: Handled at database level via unique constraints on `paytm_txn_id` . Repeated webhook dispatches exit cleanly without duplicate order generation.

Automatic Remediation: If payment succeeds but stock has depleted or outlet is closed, the system initiates an automated refund via Paytm Refund API.

Page 2 — How Paytm Connects to the Backend

Server-to-Server Protocols, Cryptographic Signatures & Entity Correlation

1. API-Level Handshake & Communication Flow

ACTUAL IMPLEMENTATION

Step 1: Frontend Request
Client calls `POST /order/checkout/paytm-session` passing `outlet_id`, items array, and delivery metadata.

→

Step 2: Backend → Paytm
FastAPI constructs parameters, formats `paytm_order_id`, signs with HMAC-SHA256, and calls Paytm Initiate API.

→

Step 3: Paytm Token Return
Paytm validates signature & returns `txnToken`. Backend returns token & MID to frontend for checkout rendering.

→

Step 4: Customer Payment
Customer completes auth at Paytm. Paytm dispatches S2S webhook to `/webhooks/paytm`.

2. Payload Details & Gateway Endpoints

PAYTM OFFICIAL

Paytm Host URLs:

- **Staging / Test:** `https://securegw-stage.paytm.in`
- **Production:** `https://securegw.paytm.in`

Initiate Transaction Endpoint:

`POST /theia/api/v1/initiateTransaction?mid={mid}&orderId={orderId}`

Critical Body Parameters Transmitted:

- `mid`: Merchant Identifier provisioned in Paytm Dashboard.
- `orderId`: Unique transaction tracking string generated by V Foods.
- `txnAmount.value` & `currency`: Stringified amount (e.g., `"250.00"`) and `"INR"`.
- `userInfo.custId`: Unique customer identifier (`user_id`).
- `callbackUrl`: Backend webhook target (`https://api.vfoods.in/webhooks/paytm`).

3. Security, Authentication & Correlation

ACTUAL IMPLEMENTATION

Cryptographic Signing (HMAC-SHA256):

- The backend utilizes `_paytm_generate_checksum()` to sign the JSON body string using the secret `PAYTM_MERCHANT_KEY`.
- Signature is transmitted in the HTTP payload header under `head.signature`.
- **Credential Isolation:** The secret key resides strictly in server environment variables (`PAYTM_MERCHANT_KEY`). It is never sent to the browser or embedded in client code.

Local Order Correlation:

- When a checkout session is initiated, `paytm_order_id` is deterministically linked to local `orders.id` via the `payments` record.
- When Paytm dispatches a webhook or status response, the backend reads `ORDERID`, fetches the row from `payments WHERE paytm_order_id = ORDERID`, and resolves the parent `order_id`.

4. Transaction Identifier & Entity Correlation Matrix

Field Name	Purpose & Description	Generated By	Stored In Database
Application Order ID	Primary UUID identifying the customer food order	V Foods Backend (FastAPI)	`orders.id`, `payments.order_id`, `refunds.order_id`
Paytm Order ID	Gateway-facing order reference (e.g. `order_abc123_169...`)	V Foods Backend	`payments.paytm_order_id`
Paytm Transaction ID	Unique financial transaction reference assigned by Paytm acquirer	Paytm Gateway Engine	`payments.paytm_txn_id` (UNIQUE)
Payment Amount	Authorized transaction amount in INR (two decimal digits)	Frontend Cart / Pricing Engine	`orders.total`, `payments.amount`
Payment Status	Financial state (`PENDING`, `SUCCESS`, `FAILED`, `REFUNDED`)	Paytm / DB Stored Proc	`payments.status`, `orders.status`
Refund Reference	Deterministic refund idempotency key (`refund-<payment_id>`)	V Foods Backend	`refunds.refund_ref` (UNIQUE)
Paytm Refund ID	Acquirer reference generated for the refund transaction	Paytm Refund Engine	`refunds.paytm_refund_id`

Security Audit Note: In strict compliance with PCI-DSS and RBI digital payment guidelines, no cardholder PAN, CVV, OTP, or merchant API secrets are stored in V Foods databases or logs. All payment authorization occurs on Paytm's regulated infrastructure.

Page 3 — Payment Status, Webhooks & Status APIs

Multi-Channel Status Verification, Server-to-Server Webhooks & Active Recovery

1. Status Communication Channels Compared PAYTM OFFICIAL

1. **Frontend Payment Response (Client-side):** User interface callback fired via JavaScript SDK upon redirect. *Inherently untrusted*. Subject to browser crashes, network drops, packet spoofing, and malicious tampering. Never used to confirm orders.
2. **Paytm Callback (Browser POST Redirect):** Browser-redirected POST to `CALLBACK_URL`. Can fail if the user closes their mobile browser or switches apps before the redirect cycle completes.
3. **Paytm Webhook (Server-to-Server Asynchronous POST):** Direct HTTPS POST from Paytm infrastructure to V Foods backend (`/webhooks/paytm`). *Authoritative channel*. Cryptographically signed, decoupled from client network health, and retried automatically by Paytm on network errors.
4. **Transaction Status API (Active Pull Mechanism):** Outbound S2S query from FastAPI to Paytm's `/v3/order/status` endpoint. Used as active reconciliation when webhooks are delayed.

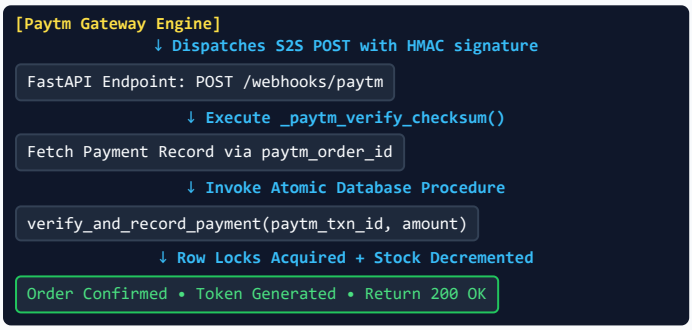
2. Why Backend Rejects Frontend Authority BEST PRACTICE

In production systems, clients are untrusted agents. Relying on frontend confirmations creates fatal attack vectors:

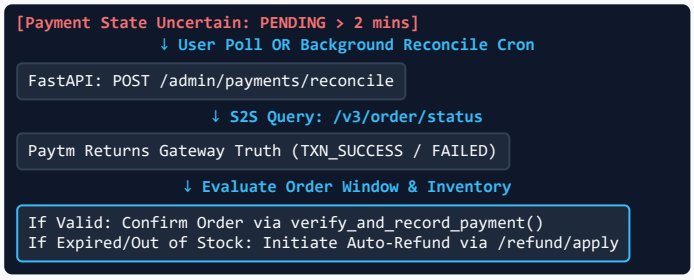
- **Man-in-the-Middle Spoofing:** An attacker can intercept client responses and inject fake 200 OK success payloads.
- **Premature Browser Termination:** If a customer closes the tab after bank debit but before the redirect fires, the order remains unpaid without S2S webhooks.
- **Dual Execution:** Flaky mobile connections can re-trigger payment callbacks, leading to duplicate database records.

V Foods Enforcement: The frontend only updates UI to "Verifying Payment..." and polls the backend until the S2S webhook confirms the order.

3. Webhook Pipeline Flowchart ACTUAL IMPLEMENTATION



4. Transaction Status API Recovery Path ACTUAL IMPLEMENTATION



5. Webhook Idempotency Guarantees ACTUAL IMPLEMENTATION

Paytm guarantees "at least once" webhook delivery. Under network congestion, Paytm will retry the identical webhook payload up to 5 times. The V Foods backend enforces strict idempotency at the database engine level via PostgreSQL:

1. Unique Transaction Key: `payments.paytm_txn_id` is constrained with a `UNIQUE` index. Attempting to insert a duplicate gateway transaction ID raises a constraint violation.

2. Stored Procedure Guard: In `verify_and_record_payment()`, the procedure first inspects `payments.status`. If already `SUCCESS`, it exits immediately and returns existing status.

3. Stock & Order Protection: Because the procedure exits early on duplicates, stock decrement logic and order confirmation events execute *strictly once*. Duplicate webhooks cannot double-decrement stock or re-credit wallets.

Page 4 — Payment Database Structure

PostgreSQL Relational Schema, Audit Ledgers, Inventory Locking & ER Topology

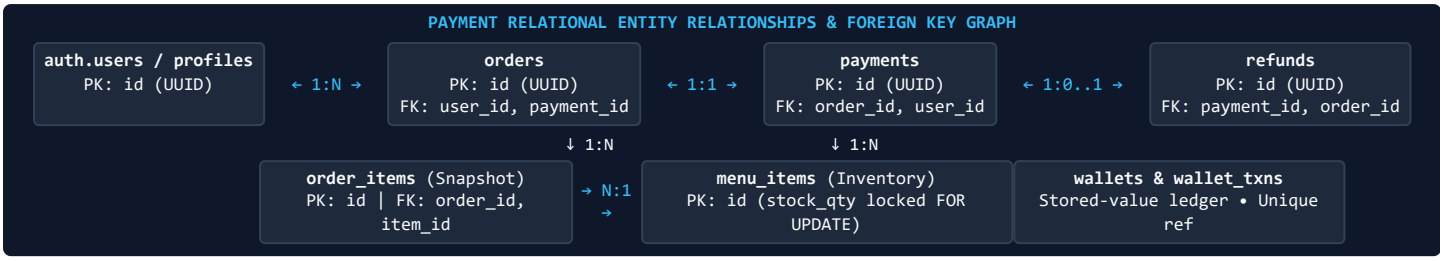


Table: payments ACTUAL IMPLEMENTATION

Primary ledger for all inbound payments (Paytm gateway, split payments, and wallet top-ups).

Column	Type	Constraint / Notes
`id`	UUID	PRIMARY KEY (Default `gen_random_uuid()`)
`order_id`	UUID	FOREIGN KEY → `orders.id` (Nullable for top-up)
`user_id`	UUID	FOREIGN KEY → `profiles.id`
`amount`	NUMERIC(10,2)	Authorized monetary amount (> 0)
`payment_purpose`	TEXT	`ORDER_PAYMENT`, `WALLET_TOPUP`, `SPLIT_PAYMENT`
`status`	TEXT	`PENDING`, `SUCCESS`, `FAILED`, `REFUNDED`
`paytm_order_id`	TEXT	Gateway-facing order reference
`paytm_txn_id`	TEXT	**UNIQUE**: Gateway reference ID
`paid_at`	TIMESTAMPTZ	Timestamp when verified by gateway
`failure_reason`	TEXT	Gateway or inventory rejection code

Table: orders ACTUAL IMPLEMENTATION

Customer food order entity linked to physical fulfillment.

Column	Type	Constraint / Notes
`id`	UUID	PRIMARY KEY
`payment_id`	UUID	FOREIGN KEY → `payments.id`
`status`	TEXT	`awaiting_payment`, `paid`, `confirmed`, `cancelled`
`total`	NUMERIC(10,2)	Gross order payable amount
`token`	INTEGER	Daily sequential pickup number for KDS
`expires_at`	TIMESTAMPTZ	Checkout window expiry (10-minute timeout)

Table: refunds ACTUAL IMPLEMENTATION

Tracks all automated and manual outbound refund transactions.

Column	Type	Constraint / Notes
`id`	UUID	PRIMARY KEY
`payment_id`	UUID	FOREIGN KEY → `payments.id`
`refund_ref`	TEXT	**UNIQUE**: Idempotency key (`refund-<payment_id>`)
`paytm_refund_id`	TEXT	Gateway refund reference
`amount`	NUMERIC(10,2)	Refunded amount in INR
`status`	TEXT	`PENDING`, `SUCCESS`, `FAILED`
`failure_reason`	TEXT	Gateway rejection diagnostic

Inventory & Wallet Tables ACTUAL IMPLEMENTATION

- `menu_items`: Contains `stock_qty`, `reserved_qty`, `available`. Evaluated with `SELECT ... FOR UPDATE` during confirmation.
- `wallets` & `wallet_txns`: Stored-value ledger for wallet orders and refunds, enforcing atomic debit/credit via unique `ref`.

Audit Gaps & Missing Tables NOT PRESENT

The following recommended audit tables are verified as absent in the current schema:

- `webhook_events`: **NOT PRESENT IN CURRENT IMPLEMENTATION**. Raw incoming webhook headers and JSON payloads are currently processed in-memory without persistent raw audit storage.
- `reconciliation_logs`: **NOT PRESENT IN CURRENT IMPLEMENTATION**. Cron execution run statistics are logged to stdout but not persisted to a dedicated relational audit table.

Page 5 — Payment Success, Failure & Refunds

Handling Uncertain Transaction States, Network Dropouts & The Automated Refund Engine

1. The Primary Real-World Failure: "The Ghost Payment" ACTUAL IMPLEMENTATION

Phase 1: Money Debited
Customer completes UPI/OTP. Paytm successfully debits ₹250 from customer bank.

→

Phase 2: Transit Failure
Mobile cellular drops, user closes browser tab, or AWS/V Foods restart drops incoming webhook.

→

Phase 3: Uncertain State
Paytm shows SUCCESS; V Foods database has payment marked `PENDING`. Customer sees no order confirmation.

2. Two-Pronged Automated Recovery ACTUAL IMPLEMENTATION

The system resolves ghost payments through two active recovery loops:

- **Path A: Paytm Webhook Retry:** Paytm's S2S notification engine monitors HTTP responses. If V Foods returned a 500 error or was temporarily unreachable, Paytm automatically re-transmits the webhook at exponential backoff intervals.
- **Path B: Active Reconciler & User Poll:** When customer views order tracker, frontend calls `POST /order/{id}/check-payment-status`. Simultaneously, background cron calls `POST /admin/payments/reconcile` querying Paytm's `/v3/order/status` API for any pending transaction older than 2 minutes.

Resolution Logic: Once Paytm returns `TXN_SUCCESS`, the backend inspects order validity. If the order is within the 10-minute window and stock is intact, the order is confirmed. If expired or unfulfillable, it triggers an automated refund.

3. The Unfulfillable Order Refund Engine ACTUAL IMPLEMENTATION

When a customer pays successfully, but the order *cannot be fulfilled* (e.g., kitchen closed, order timeout exceeded, or item out of stock):

```
[PAYMENT SUCCESS + CANNOT FULFILL]
                ↓ Backend initiates automated refund
FastAPI invokes _paytm_initiate_refund()
            ↓ Calls Paytm Refund API: POST /refund/apply
Record inserted into refunds with status PENDING
        ↓ Paytm validates and returns ACCEPTED / TXN_ACCEPTED
Paytm initiates banking reversal (1–3 banking days)
        ↓ Paytm S2S Refund Webhook: POST /webhooks/paytm/refund
Database updated to SUCCESS • Customer notified
```

4. Asynchronous Refund Semantics: "Requested ≠ Completed" PAYTM OFFICIAL

A common architectural flaw in naive payment integrations is treating a successful HTTP response from the Refund API as confirmation that the customer received money. In official Paytm payment gateway semantics:

1. PENDING / ACCEPTED
Paytm has accepted the refund request. Funds are queued for banking settlement. The merchant balance is reserved, but funds have *not yet arrived* in customer bank.

2. TXN_SUCCESS / SUCCESS
Banking partner/NPCI has successfully credited the customer account or source UPI handle. Confirmed via **Refund Webhook** or **Refund Status API**.

3. TXN_FAILURE / FAILED
The refund was rejected (e.g. beneficiary account frozen or invalid merchant balance). Requires manual intervention or secondary payout mechanism.

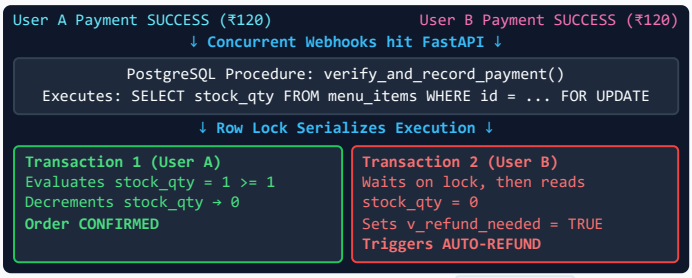
Page 6 — Refunds, Concurrency & Fault Resilience

Deterministic Keys, Rush-Hour Stock Race Conditions & Crash Recovery Across 5 Lifecycle Phases

1. Payment & Stock Concurrency Race Condition

ACTUAL IMPLEMENTATION

Scenario: Two customers pay for the *single remaining item* simultaneously via Paytm UPI.



Guarantee: PostgreSQL pessimistic row-level locking (`FOR UPDATE`) ensures that physical inventory can never drop below zero. The losing customer is automatically refunded.

2. Deterministic Keys & Timeout Handling

ACTUAL IMPLEMENTATION

Deterministic Idempotency Key:

- When invoking the Paytm Refund API, the backend calculates: `refId = f"refund-{payment_id}"`
- This unique reference is stored under `refunds.refund_ref` with a database-level `UNIQUE` index.
- If a network timeout occurs while transmitting to Paytm, retrying with the *identical* `refId` guarantees that Paytm will reject duplicate debit attempts and return the existing refund status.

Linking to Original Transaction:

- Refund requests explicitly supply `txnId: payments.paytm_txn_id` and `orderId: payments.paytm_order_id`.
- Partial refunds are supported by transmitting the calculated unfulfillable amount.

3. Server Crash Resilience Matrix Across 5 Discrete Lifecycle Phases

Failure Phase	System State & Impact	Automated Recovery Mechanism
Phase 1: Crash Before Gateway Payment	Order inserted as `awaiting_payment`; user never paid or completed transaction.	Order expires automatically after 10 minutes (`expires_at < NOW()`). No financial charge occurred.
Phase 2: Crash After Payment / Before Webhook	Customer debited at Paytm; V Foods backend offline when webhook dispatches.	**1.** Paytm webhook retry engine re-sends payload upon server restart. **2.** Background reconcile cron polls `/v3/order/status` and confirms order.
Phase 3: Crash During Webhook Execution	Backend dies during execution of stored procedure; DB transaction uncommitted.	PostgreSQL rolls back uncommitted transaction. Paytm receives no 200 OK and retries webhook. Entire procedure re-executes idempotently.
Phase 4: Crash Before Refund API Invocation	Payment verified as unfulfillable (out of stock), but server dies before calling Paytm Refund API.	Payment record remains marked `status = 'PENDING'` with `failure_reason = 'OUT_OF_STOCK'`. The reconcile cron detects stuck refund and executes `_paytm_initiate_refund()`.
Phase 5: Crash During Refund Webhook Processing	Paytm processed refund, but V Foods crashed before marking `refunds.status = 'SUCCESS'`.	Reconciliation task `POST /admin/refunds/reconcile` polls Paytm Refund Status API (`/refund/status`) and completes database update.

Page 7 — Paytm Proof & Master Architecture

Official Documentation Evidence, Visual Artifacts & Unified Master Lifecycle Architecture

1. Official Paytm Documentation Evidence & Verification PAYTM OFFICIAL



Figure 1: S2S Webhook & Callback Architecture
Paytm Payment Gateway Developer Documentation
paytmpayments.com/docs/callback-and-webhook

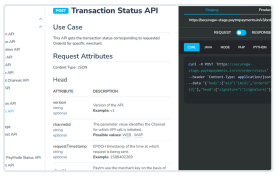


Figure 2: Transaction Status API (v3 Order Status)
Order Status Polling & Fallback Specification
paytmpayments.com/docs/api/v3/transaction-status-api



Figure 3: Automated S2S Refund API Specification
Merchant-Initiated Reversals & Unique `refId`
paytmpayments.com/docs/api/refund-api



Figure 4: Checksum Utility & Security
HMAC-SHA256 signature verification preventing payload tampering.
paytmpayments.com/docs/checksum



Figure 5: Asynchronous Refund Webhook
Gateway dispatches final settlement confirmation when funds clear.
paytmpayments.com/docs/accept-refund-webhook

2. Unified Master Payment & Refund Architecture Diagram



Architecture Verification Summary: All core checkout, signature signing, webhook handling, pessimistic row locking, and auto-refund initiation are strictly verified in active backend source code (`backend/main.py`) and PostgreSQL stored procedures (`011_paytm_three_way_split.sql` , `013_payment_reliability_and_refunds.sql`).